Machine Translated by Google

PISCINE OBJECT

Module 04 - MODÈLE DE CONCEPTION

Résumé : Ce module vous présentera le concept de modèle de conception.

Version : 1.01

Machine Translated by Google

Contenu

je Préambule 2

II Introduction 3

III Instructions générales 4

IV Exercice 00: Preparation 7

V Exercice 01: Singleton 8

VI Exercice 02 : Usine et Commandement 10

VII Exercices 03 : Médiateur 11

VIII Exercice 04 : Observer 15

IX Exercice 05: Facade 17

X Soumission et évaluation par les pairs 19

Machine Translated by Google

Chapitre I

Préambule

Avant les modèles de conception du Gang des Quatre, programmer ressemblait un peu à cuisiner pour un étudiant fauché.

Vous disposiez d'un petit arsenal de recettes simples pour préparer des plats de base, mais si vous vouliez concocter quelque chose de plus élaboré, vous deviez fouiller dans les livres de cuisine et improviser avec les ingrédients disponibles.

Les résultats étaient souvent mitigés : on pouvait obtenir un plat savoureux, mais il était souvent difficile de le reproduire ou de le modifier sans tout gâcher.

Mais heureusement, la bande des quatre a mis au point ses recettes éprouvées.

Machine Translated by Google

Chapitre II

Introduction

Le modèle de conception Gang of Four (GoF) est une collection de modèles de conception qui fournissent des solutions éprouvées aux problèmes courants rencontrés dans la conception de logiciels orientés objet.
Ces modèles ont été décrits dans le livre « Design Patterns : Elements of Reusable Object-Oriented Software » d'Erich Gamma, Richard Helm, Ralph Johnson et John Vlissides, également connus sous le nom de Gang of Four.

L'objectif principal des modèles de conception est de fournir des solutions éprouvées aux problèmes courants de conception logicielle, en réutilisant des concepts de conception déjà testés et approuvés.
Les modèles de conception ne constituent pas une solution universelle à tous les problèmes de conception logicielle, mais plutôt un ensemble de solutions aux problèmes courants qui peuvent être adaptées aux besoins spécifiques du projet.
Il est important de noter que les modèles de conception ne constituent pas une recette pour la conception de logiciels.
Elles ne remplacent pas la créativité, l'expérience et le bon sens du développeur.
Les modèles de conception sont plutôt une boîte à outils qui peut aider les développeurs à résoudre les problèmes courants de manière efficace et cohérente.

Voici quelques exemples de modèles de conception du Gang des Quatre :

- Poids plume : Partagez les objets pour économiser la mémoire.
- Méthode modèle : Définir une structure de méthode commune, mais permettre aux sous-classes de redéfinir certaines étapes.
- Adaptateur : Adapter une interface existante à une nouvelle interface attendue.
- Visiteur : Permet d'ajouter de nouvelles fonctionnalités à un objet existant sans en modifier les caractéristiques structure.
- Commande : Encapsuler une action sous forme d'objet, permettant ainsi la gestion de cette action. à paramétrer.
- Constructeur : Séparer la construction d'un objet complexe de sa représentation afin que le même processus de construction puisse créer différents types de représentations.
- Pont : Découpler une abstraction de son implémentation afin qu'elles puissent varier indépendamment.

En apprenant et en utilisant les modèles de conception du Gang of Four, les développeurs peuvent améliorer la qualité, la maintenabilité et la réutilisabilité de leur code.
Les exercices pratiques constituent un excellent moyen de consolider votre compréhension des concepts de modèles de conception et de les appliquer à des situations réelles.

Machine Translated by Google

Chapitre III

Instructions générales

Dans ce module, vous devrez implémenter un ensemble de patrons de conception. Certains nécessiteront l'utilisation de modèles, d'autres non. Certains vous feront recourir à l'héritage et au polymorphisme, d'autres non.
Si vous n'êtes pas familier avec ces sujets, vous devriez revoir vos bases en C++, car vous en aurez besoin ici.

Dans ce module, si vos méthodes non paramétrées ne dépassent pas 3 instructions, vous êtes autorisé à l'implémenter directement dans le fichier d'en-tête, comme suit :

```
classe Foo
____

{ privé : int
        _valeur; char *_chaîne;


        void _initializeString(); // Méthode trop longue pour être placée dans l'en-tête
publique:
        Foo() : //Constructeur avec 0 instruction _value(0),
                _string(nullptr)

        {

        }
        Foo(char *p_string); // Méthode trop longue pour être placée dans l'en-tête const int&
        value() const // Accesseur simple, sans vérification, retournant uniquement la valeur {

                renvoyer (_valeur);
        }
        const char *string() // Getter avec un appel à la méthode d'initialisation de variable {

                si (_string == nullptr)
                        _initializeString(); retourner
                (_string);
        }
        const char *string() const //Le getter const n'est pas autorisé à initialiser une variable {

                si (_string == nullptr)
                        throw std::runtime_error("Aucune chaîne initialisée"); return (_string);

        }
}
```

Machine Translated by Google

Les instructions ne correspondent pas à des lignes. Une instruction est une unité de code.

- La boucle conditionnelle est une instruction
- Un appel de méthode correspond à une instruction
- Un retour correspond à une instruction.

Tout dépassement du nombre d'instructions dans l'en-tête entraînera une erreur.
instantané 0.

Les méthodes génériques sont évidemment exclues de la limitation à 3 instructions, puisqu'elles ne peuvent pas être placées dans des fichiers .cpp. Mais n'utilisez pas de méthodes génériques uniquement pour éviter la création de fichiers .cpp !

N'oubliez pas d'appliquer l'encapsulation à votre code afin de garantir la protection adéquate de vos classes et une gestion optimale de l'accès aux variables. Il s'agit d'un élément crucial pour la réussite de ce projet.

exercice.

Veillez à bien définir la propriété de chaque attribut en établissant un lien.
Chaque attribut associé à sa classe propriétaire. Cela permettra une gestion efficace.
la gestion des ressources système, qui est essentielle au succès de
cet exercice.

Machine Translated by Google

/! NE RÉPÉTEZ PAS VOTRE CODE /!

Si vous réécrivez exactement le même code à quelques détails près, vous devez trouver un moyen de le factoriser. Sinon, vous obtiendrez la note de 0 à l'évaluation.

/! NE RÉPÉTEZ PAS VOTRE CODE /!

Ce sujet étant axé sur un code propre et facile à lire, vous êtes

Il est permis de modifier tout élément tel que le type, la signature, le nom, etc., à condition de respecter l'esprit de la fonctionnalité. Par exemple, si une méthode exige le retour d'un `std::vector` de `Student*`, cela signifie que vous devez retourner un conteneur, stockant des éléments mais n'en étant pas propriétaire.

Étudiant*. Vous êtes autorisé à modifier ceci pour renvoyer une std::list de

std::shared_ptr Étudiant.

Machine Translated by Google

Chapitre IV

Exercice 00: Preparation

	Exercice 00
Exercice 00 : Préparation scolaire	
Répertoire de dépôt : ex00/	
Fichiers à soumettre : ex00/*.cpp, ex00/*.hpp Fonctions	
autorisées : conteneurs et structure STD standard	

Dans cet exercice, vous devez ouvrir le dossier de pièces jointes, où vous trouverez un fichier datas.hpp contenant plusieurs structures, classes et quelques énumérations que nous avons gentiment pré-créées pour vous !

Vous devez le lire et le décomposer au moins en plusieurs fichiers .hpp.

Une fois que vous êtes satisfait de la répartition de vos énumérations/classes/structures, vous devriez Passez à l'exercice suivant !

Machine Translated by Google

Chapitre V

Exercice 01: Singleton

	Exercice 01
Exercice 01: Singletons	
Répertoire de dépôt : ex01/	
Fichiers à remettre : ex01/*.cpp, ex01/*.hpp, ex01/singetons.hpp, ex01/main.cpp Fonctions autorisées :	
conteneurs et structure STD standard	

Vous devez créer plusieurs objets afin de stocker chaque élève, chaque membre du personnel, chaque cours et chaque salle de votre école simulée.

Vous devez créer les objets suivants :

- Liste des élèves, contenant tous les élèves inscrits dans l'établissement.
- Liste du personnel, contenant tous les membres du personnel de l'école.
- Liste des cours, qui répertorie tous les cours actuellement dispensés dans l'établissement.
- Liste des salles, contenant toutes les salles de l'établissement.

Ces objets devraient comporter au moins une méthode permettant à l'utilisateur d'accéder à l'objet, une méthode pour ajouter des éléments à l'intérieur de ces objets et une méthode pour les récupérer.

Ces objets nouvellement créés ne doivent exister qu'une seule fois pour tous les threads possibles du programme, et doivent pouvoir être joints depuis n'importe quel endroit où ils ont été inclus.

/! NE RÉPÉTEZ PAS VOTRE CODE /!
N'oubliez pas ce que nous avons dit lors de la partie instructions générales : répéter votre code donnera un résultat de 0.
L'ajout de la fonctionnalité de modèles au C++ n'est pas dû à un hasard.
/! NE RÉPÉTEZ PAS VOTRE CODE /!

Machine Translated by Google

Vous devriez créer une fonction principale dans laquelle vous pourrez instancier ces deux objets, les remplir manuellement en insérant de nouveaux éléments dans chacun des objets créés, et pouvoir les récupérer.

Le nom de cet exercice n'est probablement pas là juste pour faire joli, vous devriez peut-être le rechercher sur Internet !

Machine Translated by Google

Chapitre VI

Exercice 02 : Usine et commandement

	Exercice 02
Exercice 02 : Fabrique et commande	
Répertoire de dépôt : ex02/	
Fichiers à déposer : ex02/*.cpp, ex02/*.hpp, ex02/main.cpp Fonctions	
autorisées : conteneurs et structure STD standard	

Les directeurs et les secrétaires sont des collaborateurs qui travaillent main dans la main : le secrétariat crée des formulaires que le directeur reçoit et remplit, une fois qu'ils ont été complétés avec les informations obligatoires concernant leur contenu.

Par conséquent, vous êtes prié de créer le système qui permettra au Secrétaire de créer le formulaire. en fonction de l'énumération FormType, elle recevra en entrée la méthode createForm.
Une fois le formulaire établi par la secrétaire, vous devrez le remplir avec les informations requises, puis le remettre au directeur pour signature et validation.

Vous devez également remplir les 4 types de formulaires que nous avons pré-créés pour vous.
À cette fin, nous vous conseillons vivement de :

- Ajoutez tous les attributs nécessaires à l'intérieur de ces formulaires.
- Ajoutez une méthode quelconque dans ces deux formulaires pour renseigner les nouveaux attributs avec des données.
- Créez la méthode execute pour les nouveaux formulaires, afin de leur permettre de s'exécuter là Proposer une fois que le directeur l'aura acceptée.

Vous êtes autorisé à modifier le formulaire ou toutes les classes dérivées si vous constatez qu'il manque des éléments.

Une fois ces formulaires créés, vous devez créer un programme principal dans lequel vous en instanciez certains, les donnez ou non au directeur, et essayez de les exécuter, pour constater que lorsqu'ils sont signés, ils ont un effet, et que lorsqu'ils ne le sont pas, ils ne sont pas exécutés correctement.

Machine Translated by Google

Chapitre VII

Exercice 03: Mediator

	Exercice 03
Exercice 03: Mediator	
Répertoire de dépôt : ex03/	
Fichiers à soumettre : ex03/*.cpp, ex03/*.hpp, ex03/main.cpp Fonctions	
autorisées : conteneurs et structure STD standard	

Maintenant que nous avons créé ces formulaires, il est temps de donner vie à cette école !
C'est le rôle du directeur : il est chargé de donner des cours aux professeurs et d'affecter les étudiants aux cours !

L'idée est assez simple : tout doit passer par le directeur, qui devra signer chaque action se déroulant dans l'établissement.

Peut-être que cette affirmation a un lien avec le nom de l'exercice, je n'en suis pas sûr...

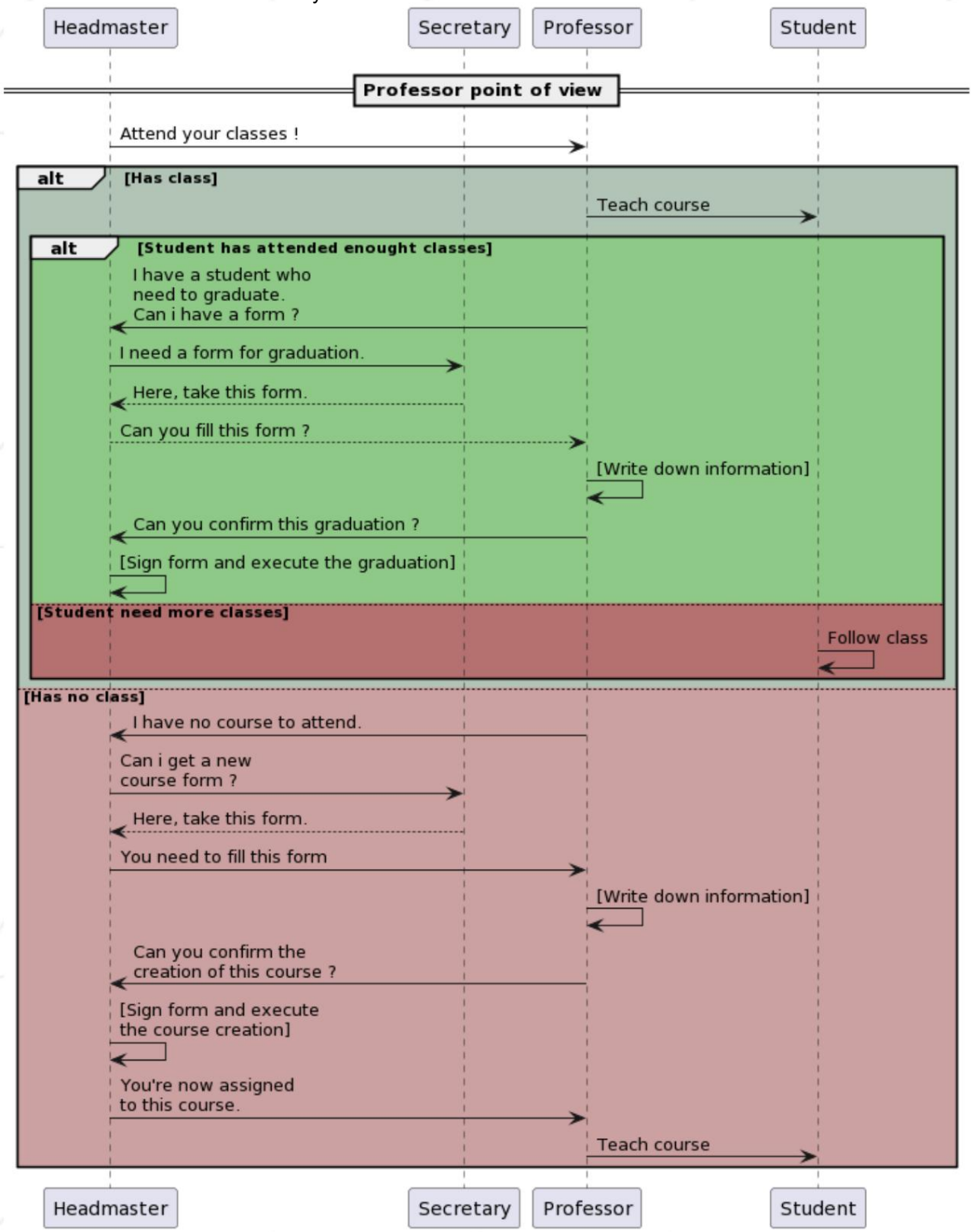
Pour chaque action, il exigera un formulaire des secrétaires. Une fois que le demandeur aura…
Remplissez le formulaire, il le signera et exécutera les formalités requises.

Vous devez avoir suivi le cours de directeur d'école pour pouvoir adopter ce comportement.
Vous devez également mettre en place un système permettant aux professeurs de créer de nouveaux cours et aux étudiants de s'y inscrire.
Nous vous donnerons un exemple d'interaction entre le directeur, la secrétaire, le professeur et l'élève, afin d'illustrer l'interaction que nous attendons de vous.

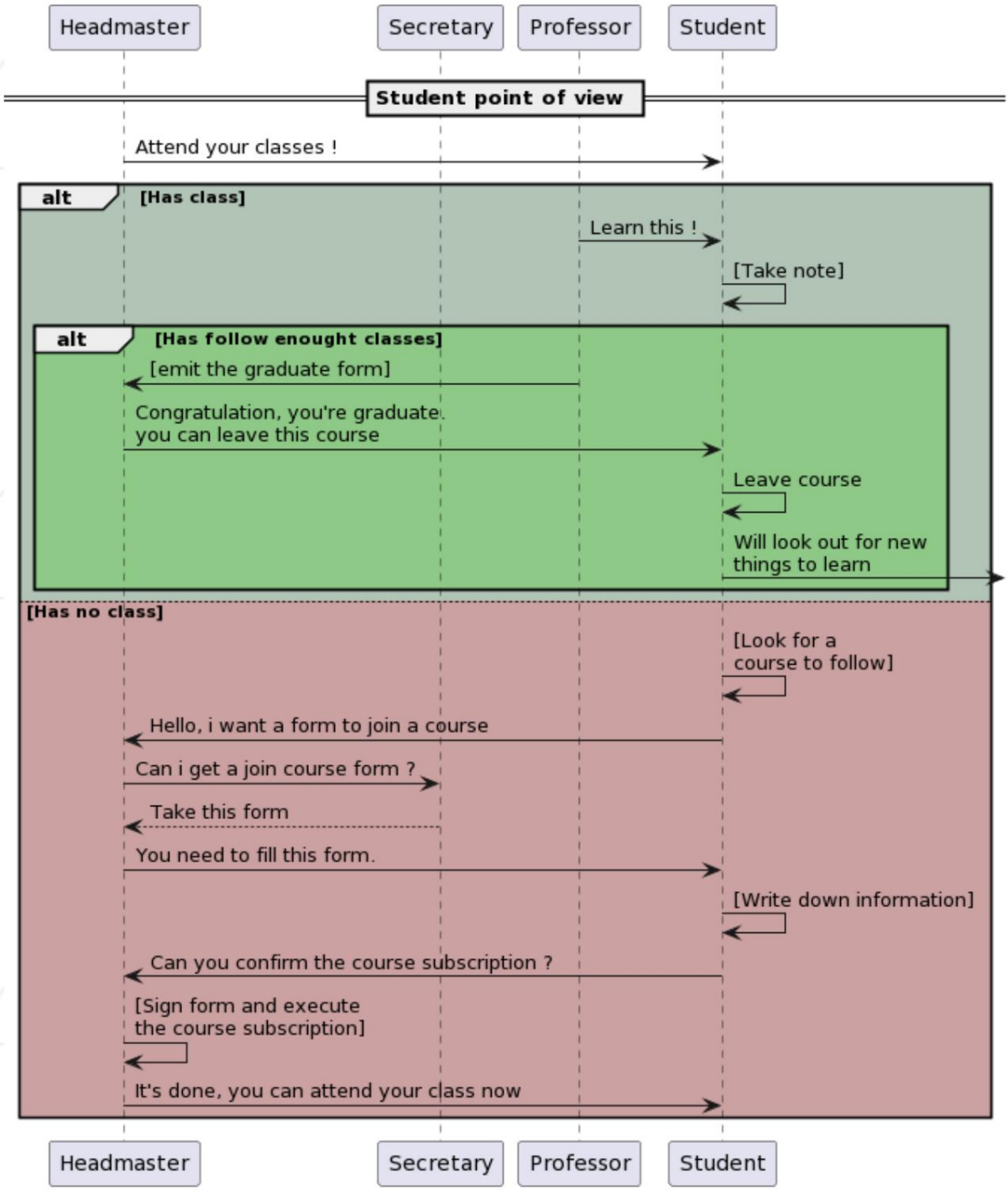
Il est présenté comme un dialogue entre eux, mais n'hésitez pas à créer une séquence de dialogue « réelle ».
gram, en interprétant ce diagramme !

Machine Translated by Google

Vous devez mettre en œuvre les systèmes suivants :



Machine Translated by Google



Machine Translated by Google

Une fois ces deux types d'interactions terminés, vous devez créer le dernier type de formulaire : la création de la salle !

L'idée est assez simple : si, en voulant donner un cours, un professeur ne trouve pas de salle de classe libre, il pourra demander au directeur d'en créer une comme par magie !

Essayez de deviner comment cela va se faire, en vous basant sur les deux derniers exemples.

Vous devez fournir un programme principal qui démontre les fonctionnalités suivantes :

- Le directeur peut exiger que les professeurs assistent à leurs cours.
- Si un professeur n'a pas de cours, il faut lui en attribuer un nouveau.
- Si un professeur ne trouve pas de salle de classe vide, il doit s'en voir attribuer une nouvelle. un.
- Les étudiants qui ne sont pas inscrits à un cours tenteront de s'y inscrire.
- Une fois que l'élève a suivi le nombre de cours requis, il reçoit son diplôme des mains du directeur, sur recommandation du professeur responsable du cours.

Machine Translated by Google

Chapitre VIII

Exercice 04: Observer

	Exercice 04
Exercice 04: Observer	
Répertoire de dépôt : ex04/	
Fichiers à soumettre : ex04/*.cpp, ex04/*.hpp, ex04/observer.hpp, ex04/main.cpp Fonctions autorisées :	
conteneurs et structure STD standard	

Maintenant que le directeur peut imposer des cours aux professeurs et aux élèves, ils vont s'épuiser au travail ! Il faudra peut-être leur accorder des temps libres réguliers. Pour cela, il faudra simuler un système de sonneries.

L'idée est assez simple : chaque fois que les étudiants ou les professeurs entendent la sonnerie, ils sont autorisés à quitter la classe.

Une fois leur temps libre écoulé, la cloche sonnera à nouveau et ils retourneront en classe.

Ce sera au directeur de sonner cette cloche !

Un enum vous a été fourni dans le fichier d'entrée, l'enum Event, qui représentera cet événement de sonnerie de cloche.

Vous devez créer une classe où les étudiants et le professeur pourront être avertis lorsque la cloche sonne.

Comme toujours, le nom de l'exercice n'est pas là par hasard, il pourrait vous être utile !

Vous DEVEZ faire en sorte que la sonnerie indique aux étudiants et aux professeurs le début ou la fin de la pause. Si les professeurs ou les étudiants doivent vérifier si la pause est terminée, la note sera de 0.

Machine Translated by Google

Vous devez fournir un fichier principal montrant comment fonctionne votre système.

Machine Translated by Google

Chapitre IX

Exercice 05: Facade

	Exercice 05
Exercice 05: Facade	
Répertoire de dépôt : ex05/	
Fichiers à soumettre : ex05/*.cpp, ex05/*.hpp, ex05/main.cpp Fonctions	
autorisées : conteneurs et structure STD standard	

Maintenant que votre école semble fonctionner correctement, avec le directeur qui fait son travail, les professeurs qui enseignent, les secrétaires qui font leur travail de secrétaire et les élèves qui apprennent, il est temps de créer une classe pour englober tout ce processus complexe !

Cela se fera dans une nouvelle classe, appelée École, que vous devez créer.

Cette nouvelle classe doit contenir une méthode permettant d'effectuer les actions suivantes :

- void runDayRoutine() : exécute la routine de la journée scolaire, composée de :

  1. classes de lancement
  2. Autoriser les étudiants et les professeurs à participer à des activités récréatives
  3. classes de lancement
  4. Déjeuner de lancement
  5. classes de lancement
  6. Autoriser les étudiants et les professeurs à participer à des activités récréatives
  7. classes de lancement

- void launchClasses() : Permet au directeur de demander aux élèves et aux professeurs d'assister aux cours.

Machine Translated by Google

- void requestRingBell() : Faire sonner la cloche par le directeur
- void recruteProfessor() : Ajouter un nouveau professeur à l'établissement
- void recruteStudent() : Ajouter un nouvel élève à l'établissement
- Course* getCourse(std::string p_name) : renvoie un cours portant le nom donné
- std::vector<Student*> getStudents() : renvoie un vecteur contenant tous les étudiants dent à l'école
- std::vector<Professor*> getProfessors() : renvoie un vecteur contenant tous les professeurs de l'établissement
- Headmaster getHeadmaster() : renvoie le directeur de l'école
- Secretary getSecretary() : renvoie le secrétaire de l'école
- void graduationCeremony() : lance la remise des diplômes de tous les étudiants qui le peuvent obtenir son diplôme au moment souhaité

Comme vous l'avez peut-être deviné, vous devrez peut-être modifier la façon dont nous avons fabriqué les choses. Auparavant ! Vous pouvez ajouter un nouvel événement, modifier la façon dont les gens obtiennent leur diplôme, je ne sais pas !

Machine Translated by Google

Chapitre X

Soumission et évaluation par les pairs

L'idée est de vous obliger à RÉFLÉCHIR à ce que vous faites, et non pas simplement à suivre des règles sans bien comprendre pourquoi elles existent.
C'est précisément pour cette raison que vous serez interrogé sur tout ce que vous avez fait, alors réfléchissez-y et soyez prêt à discuter des choix que vous avez pu faire en matière d'architecture ou de mise en œuvre !

Veuillez déposer votre travail dans votre dépôt Git comme d'habitude. Seul le contenu de votre dépôt sera évalué lors de la soutenance. N'hésitez pas à revérifier les noms de vos dossiers et fichiers pour vous assurer de leur exactitude.